

Mise en conformité — Bonnes pratiques d’optimisation (Étapes 6.1 → 6.7)

Ce document récapitule **tout** ce qui a été mis en place pour conformer le projet Lab Scheduling Automation (Loyola) au processus d’optimisation décrit dans l’audit interne (`AUDIT_OPTIMISATION_COMPLET`). Chaque étape est traitée de A à Z, avec les fichiers concernés et la façon de la vérifier.

Méta	Valeur
Date	2026-06-18
Branche	<code>feature/optimization-conformance</code>
Tests	24 / 24 au vert (<code>pytest tests/</code>)

Tableau de synthèse

Étape	Objectif	Statut	Livrables
6.1	Formaliser le problème	✓	docs/PROBLEM-FORMULATION.md (versionné)
6.2	Séparer données/ modèle + couche QA	✓	data_quality.py + intégration pipeline.py
6.3	Construire par incréments + tests de non-régression	✓	tests/ (24 tests)
6.4	Diagnostic d'infaisabilité	✓	diagnose_infeasibility() dans pipeline.py
6.5	Réglage & reproductibilité du solveur	✓	configure_solver(), add_week_hints(), graine fixe
6.6	Mesure de la qualité (KPIs)	✓	kpi_report.py + intégration pipeline.py
6.7	Industrialisation (CI, repro, jointure versionnée)	✓	.github/workflows/ci.yml, requirements-dev.txt, build_master_schedule.py, .gitignore

6.1 — Formaliser le problème

Document de référence **versionné** docs/PROBLEM_FORMULATION.md : ensembles, paramètres, variables de décision, contraintes dures **C1/C4/C5**, contraintes souples (pénalités first_excess=100, last_deficit=100, gap=200, parité=50, salle réservée=100 000) et fonction objectif. Il clarifie surtout l'**architecture en 2 étages** (glouton fige jour/bloc/salle → CP-SAT décide la semaine), donc le sens exact du mot « OPTIMAL ».

6.2 — Données / modèle séparés + couche qualité (QA)

Nouveau module `data_quality.py`, sans effet de bord, qui :

- vérifie l'intégrité du master (colonnes critiques, IDs vides) ;

- **réconcilie la jointure** aulario $\bowtie$ alumnos et chiffre les orphelins ;
- **réconcilie la formation des groupes** (anti-fuite « overflow ») : inscrits vs placés vs non placés, par matière.

Validé sur données réelles : reproduit la perte **969 → 938 (31 étudiants, 3,2 %)** identifiée dans l'audit. Intégré dans `run_pipeline` (non bloquant par défaut, mode `strict=True` pour CI/tests). Sorties : `reports/data_quality_report.{json,txt}` .

6.3 — Incréments + tests de non-régression

Suite `tests/` (pytest), 24 tests, exécutée en < 0,5 s **sans** lancer le solveur complet :

- `test_data_quality.py` — intégrité, réconciliations, mode strict ;
- `test_kpi_report.py` — KPIs, robustesse aux entrées vides ;
- `test_solver_config.py` — paramétrage, warm-start, diagnostic, mini-résolution CP-SAT ;
- `test_problem_constants.py` — cohérence des constantes documentées en 6.1.

6.4 — Diagnostic d'infaisabilité

`diagnose_infeasibility()` : au lieu d'un simple « INFAISABLE », identifie les couples (salle|matière, jour, bloc) **sur-saturés** (plus de séances que de semaines disponibles) et écrit `reports/infeasibility_S<n>.txt` . Appelé automatiquement quand le solveur échoue, avant le repli (recovery).

6.5 — Réglage & reproductibilité du solveur

`configure_solver()` centralise pour **tous** les appels (principal + repli) : `random_seed = 42` , `relative_gap_limit = 0.02` , `max_time_in_seconds` , `log_search_progress` . `add_week_hints()` fournit un **démarrage à chaud** (semaines régulièrement espacées, hints non contraignants). Résultats désormais **reproductibles** d'une exécution à l'autre.

6.6 — Mesure de la qualité (KPIs)

Nouveau module `kpi_report.py` : à chaque exécution, calcule % de **placement**, nb de groupes / overflow / récupérés, distribution des tailles, **équilibre par jour** (suivi du goulot du vendredi), **occupation des salles**, et statistiques **solveur** (statut, objectif, gap, temps). Sorties : `reports/kpi_report.{json,txt}` . On ne se contente plus d'un « ça tourne ».

6.7 — Industrialisation

- **CI GitHub Actions** `.github/workflows/ci.yml` : sur push/PR, installe les dépendances, vérifie la syntaxe et lance `pytest` (Python 3.11 et 3.12).
- **requirements-dev.txt** : dépendances de test reproductibles.
- **build_master_schedule.py** : script de jointure **versionné et instrumenté QA** (clé MixtoID), avec avertissement documenté sur les règles de normalisation horaire amont non fournies (pas de reconstruction « octet pour octet » inventée).

- `.gitignore` : exclut caches et sorties générées (`reports/`).

Comment vérifier

```
# 1) Tests de non-régression (rapide, sans solveur)
pip install -r requirements-dev.txt
pytest tests/ -v

# 2) Régénérer/diagnostiquer la jointure (QA)
python build_master_schedule.py \
    --alumnos data_clean/report_AlumnosGruposCentroDecanos.xlsx \
    --aulario data_clean/revisionAulario.xlsx \
    --out data_clean/master_schedule_rebuilt.csv

# 3) Pipeline complet -> génère aussi reports/data_quality_report.* et reports/
kpi_report.*
python pipeline.py
```

Tous les nouveaux modules sont **défensifs** (ils ne cassent jamais le pipeline : try/except, non bloquants par défaut) et **testables isolément**.

Journal des versions

Version	Date	Changement
1.0.0	2026-06-18	Mise en conformité initiale des étapes 6.1 → 6.7.